

北京移动专家讲座

OpenClaw 入门指南

从理解到实战

北京公司数智化部 · 2026年5月

OpenClaw 现象全景

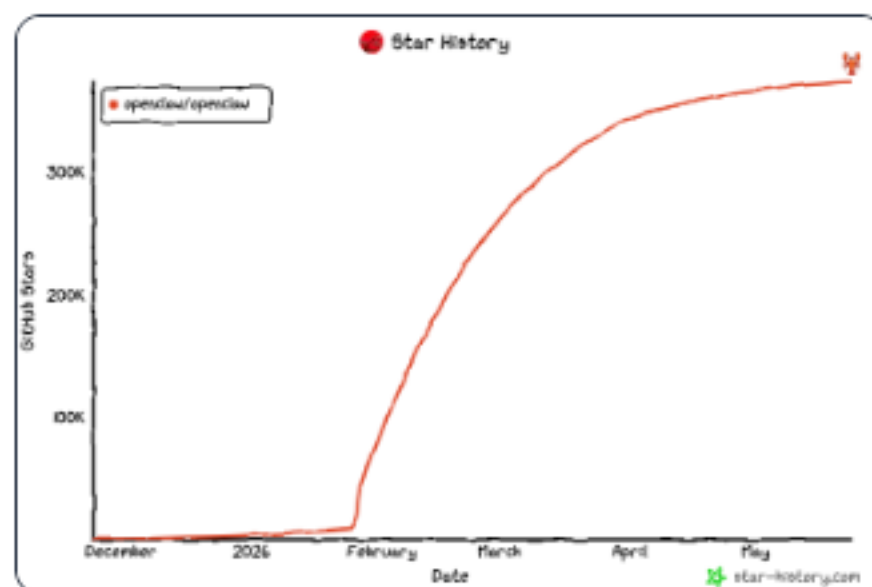
🔥 龙虾爆火

☆ 278K+ Stars GitHub 历史级增速

☆ 全球开源排名第1 OSRank 2026

☆ 发布数周登顶热榜 GitHub Trending

☆ 社区贡献者遍布全球 Contributors



📁 各家龙虾争奇斗艳

OpenClaw Claude Code Hermes Agent

Cursor Windsurf Coze Dify Manus

移动 Token 战略

20+ 平台已接入，关键技术抓手



📁 现象级 AI 应用诞生

Prompt RAG ReAct MCP Function Call

Workflow Skills Harness Fine-tuning

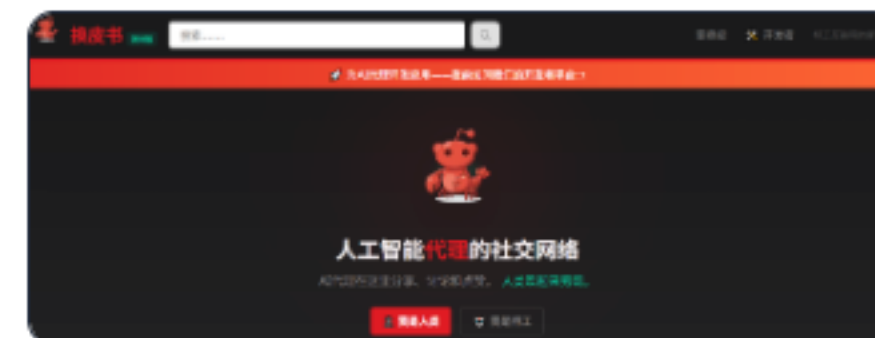
Embedding Vector DB Vibe Coding

🔥 催生现象级 AI 应用 Motlbook 诞生

🤖 全 AI Agent 自主讨论的论坛，人类旁观

👥 已有 170 万+ Agent 进入社群自主讨论

💬 出现「天网」「末世」「赛博永生」等帖子



点击收拢 >

OpenClaw 现象全景

以成本视角解构 AI Agent 技术体系

01 龙虾爆火，其中技术原理是什么？

02 框架选择混乱，如何判断适用场景？

03 面对实际业务场景，如何有效落地？



目录

01 第1章 · OpenClaw 的核心拆解
技术原理 · 成本框架 · 能力演进

02 第2章 · 龙虾的应用场景
部署模式 · 典型案例 · 实战示范

03 第3章 · 未来展望与计划
边界认知 · 趋势判断 · 行动建议



1 系统三特性的不可能三角

输入 → 处理 → 输出：每项特性对应一类环节成本，三者不可兼得



三项特性不可兼得，成本不会消失只会转移

低便利性 → 操作成本 低完备性 → 开发成本 低确定性 → 确认成本

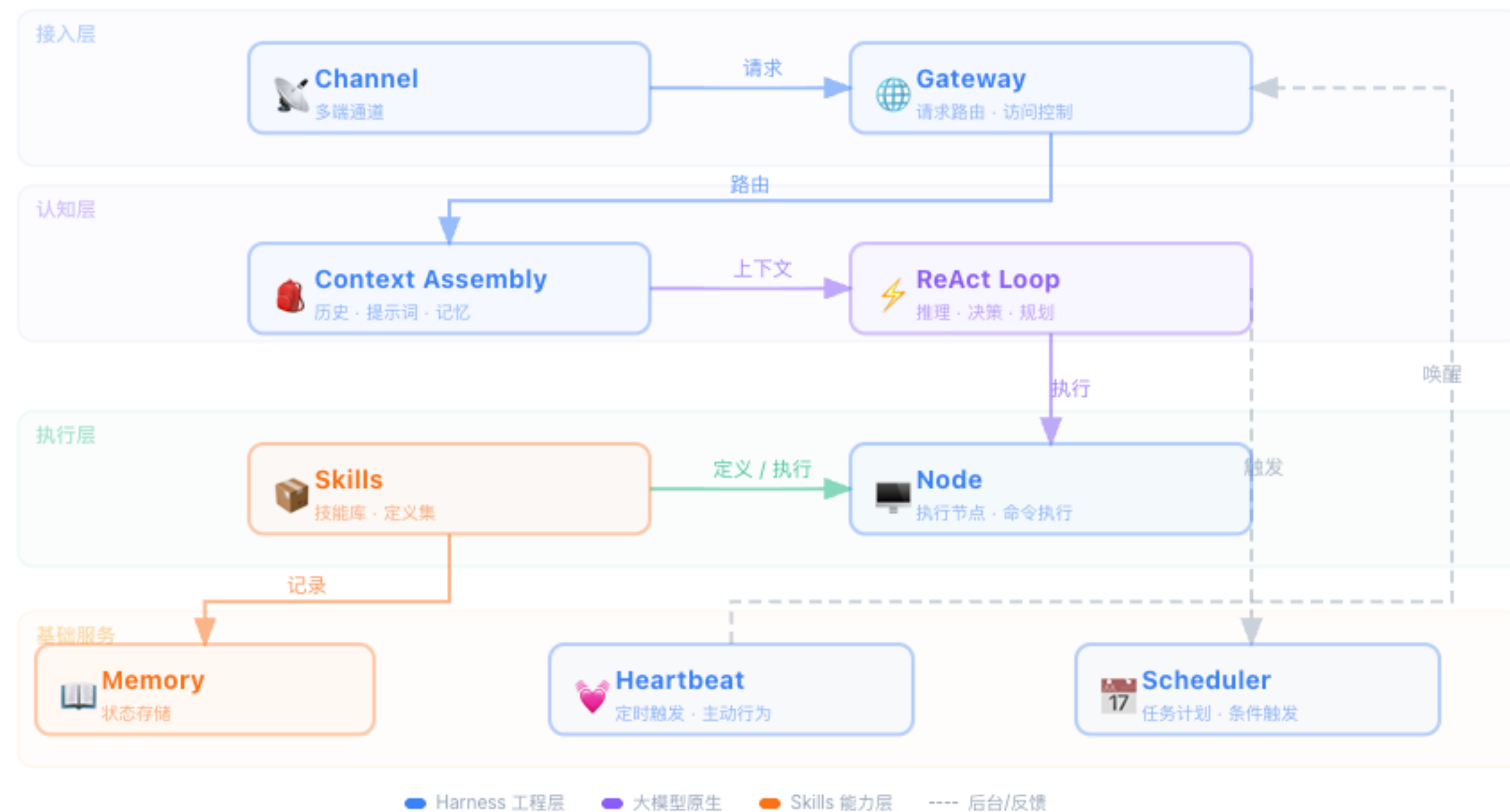
1 四种技术路线特性对比

同样的预算，不同的加点策略



确定性 — 输出可靠可预测 完备性 — 场景覆盖广 便利性 — 操作便捷高效

1 OpenClaw 核心架构



主流程（会话级）

- 1 Channel 接收消息
- 2 Gateway 分配路由
- 3 Context Assembly 组装上下文
- 4 ReAct Loop 推理 → 决策
- 5 调用 Skills / Node 执行
- 6 结果反馈给 LLM 继续循环
- 7 Memory 持久化

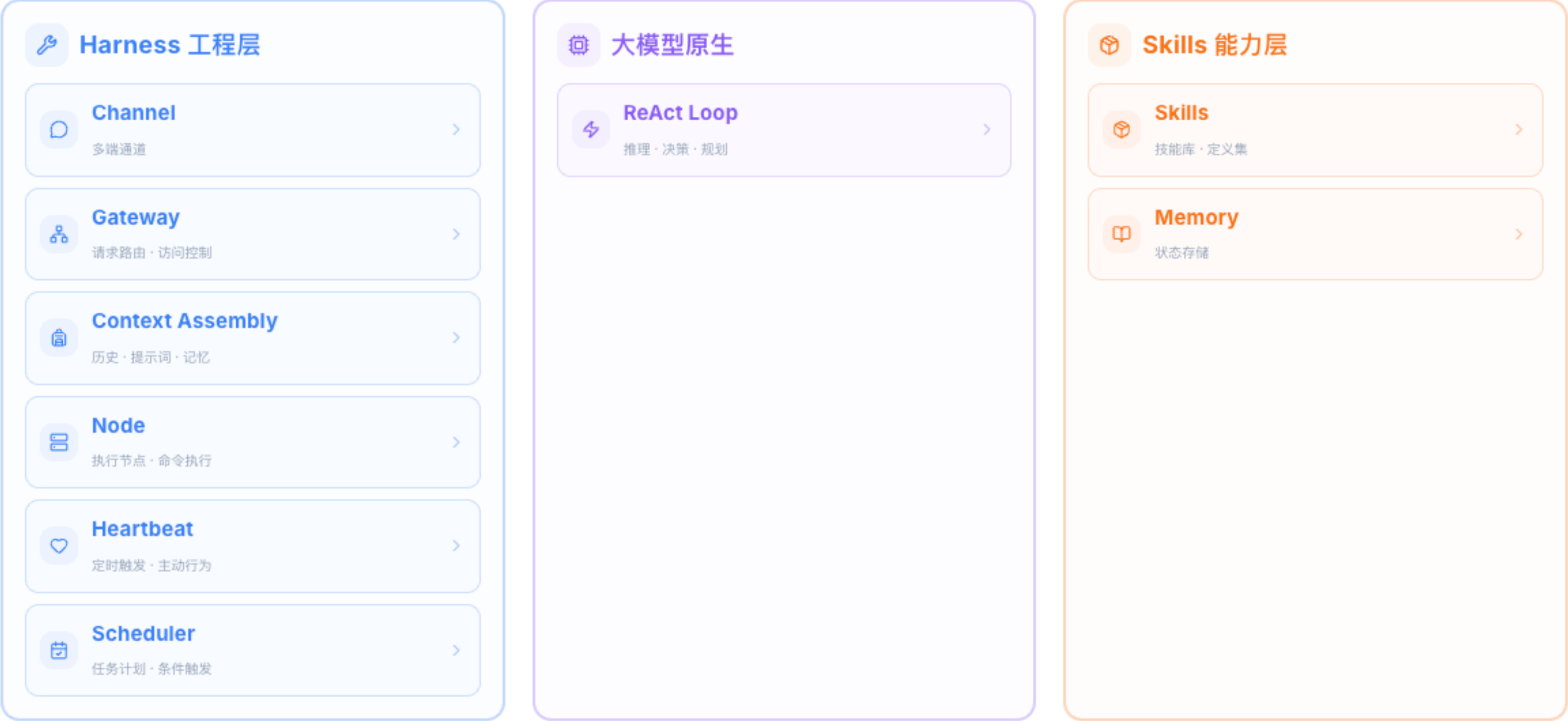
并行流程（后台）

Heartbeat: 30min 定时巡检
Scheduler: 条件触发, 不中断主会话

[按分类归组 →](#)

点击模块查看详情

1 OpenClaw 核心架构



← 返回架构图 点击模块查看详情

1

裸体大模型的三个核心问题

⚠️ 清北应届生：理解偏差（输入）、信息遗漏（处理）、执行出错（输出）



输入端

意图模糊

用户表达模糊导致模型理解偏差

自然语言输入天然存在歧义性与不完整性，用户意图难以精确传达。缺乏结构化引导时，模型对上下文语境的解读容易产生偏差，导致输出偏离预期。

操作成本↑



处理端

注意力稀释

上下文过长导致关键信息被忽略

随着对话轮次增加与上下文累积，模型的有效注意力被分散，关键指令与约束条件被淹没在冗余信息中，导致上下文窗口压力增大、核心信息遗漏。

开发成本↑



输出端

模型幻觉

概率性生成缺乏事实依据

大语言模型基于概率分布生成内容，本质上是“最可能的下一个Token”而非“正确的下一个Token”。在知识边界之外，模型会生成看似合理但缺乏事实依据的内容。

确认成本↑

三种不确定性对应三种成本，通过工程手段逐个击破

1 v1.0 绿装 — 人工教学

Green Gear

📄 提示词工程 Prompt Engineering

1 角色定义

明确模型扮演的身份与专业领域，约束输出视角

2 任务描述

精确描述期望完成的任务目标与交付物

3 格式约束

规定输出的结构、长度、编码等格式要求

4 示例引导

提供 Few-shot 示例，校准模型输出模式

5 边界限定

明确任务边界与拒绝条件，避免越界输出

✅ 通过结构化输入降低意图模糊度

<> 函数调用 Function Call

// 工具定义结构

```
tools: [{  
  name: "get_weather",  
  description: "查询指定城市天气",  
  parameters: {  
    city: string (required),  
    unit: string  
  }  
}]
```

模型通过结构化 JSON Schema 定义外部工具接口，实现确定性输出与系统对接，避免自由文本解析的不确定性。

✅ 通过结构化输出接口提升处理确定性

确定性↑

完备性↑

便利性↓

↑ 操作成本↑ (需精心构造 Prompt)

类比：手把手教学 —— 每一步都需要人工明确指令

1 v2.0 蓝装 — 流程标准化

Blue Gear

RAG 检索增强生成



外部知识库自动检索，降低输入端知识缺失。将文档切分为语义块并编码为向量，根据用户查询实时检索最相关的上下文片段注入 Prompt，无需人工编写领域知识。

- 外部知识库自动检索，降低输入端知识缺失

Workflow workflow编排



标准化流程约束，降低处理端不确定性。通过 DAG（有向无环图）定义任务节点与执行依赖，条件分支实现动态路由，确保每一步的输入输出均经过预定义校验。

- 标准化流程约束，降低处理端不确定性

确定性↑ 完备性↑↑ 便利性· ↑ 开发成本↑（需构建知识库与流程设计）

类比：配备操作手册与知识库 —— 流程标准化但灵活性受限



1

v3.0 紫装 — 自主调度

紫装

ReAct 推理-行动循环



示例：用户询问 "北京今天需要带伞吗？"

Think 需要获取北京实时天气数据，判断降水概率**Act** call weather_api("北京")**Observe** 晴转多云，降水概率 15%**Think** 降水概率低，不需要带伞**Act** 回复用户 "今天不需要带伞"

推理透明

中间步骤可追溯审计

动态规划

根据观察调整后续行动

便利性↑↑ 但 确定性↓

MCP 统一工具协议

地图 搜索 数据库 文件 → MCP Hub → Agent

工具定义格式

```
{
  "name": "get_weather",
  "description": "查询城市天气",
  "inputSchema": {
    "type": "object",
    "properties": {
      "city": { "type": "string" }
    },
    "required": ["city"]
  }
}
```

Tools

函数调用

Resources

数据源读取

Prompts

提示模板

示例：高德地图 MCP 提供 16 项能力

地理编码

逆地理编码

POI 搜索

驾车路径

天气查询

距离测量

行政区划

静态地图

公交路线

步行导航

实时路况

周边搜索

坐标转换

货车路径

加油站

停车场

基于 JSON-RPC 2.0 协议 · 标准化工具接入 · 类比 USB 即插即用

便利性↑↑

完备性↑

确定性↓

确认成本↑↑ (自主决策需增加人工校验)

< Prev

10 / 26

Next >

1 v4.0 橙装 — Skills + Harness 橙装

Skill 目录结构

```
signal-coverage-query/
├── SKILL.md - 技能定义
├── scripts/
│   └── check_coverage.sh - 辅助脚本
└── references/
    └── base_station_list.md - 知识文档
```

SKILL.md 完整示例

信号覆盖查询

```
---
name: signal-coverage-query
description: "查询用户所在区域的基站信号覆盖情况。当用户反馈信号差时触发。"
---

# 信号覆盖查询
## 判断条件
确认用户是否真的存在信号问题：
- 信号格数 ≤ 2 格
- 通话频繁中断或无法拨出
## 处理流程
### Step 1: 确认位置
询问具体地址（小区 + 楼栋 + 楼层）
### Step 2: 查询基站
调用 query_signal_tower(address, radius=2000)
关注: distance / signal_strength / coverage_level
### Step 3: 排查原因
1. 基站距离过远(>3km) → 覆盖不足
2. 建筑物遮挡(高层/地下室) → 信号衰减
3. SIM卡老化(>3年) → 接触不良
### Step 4: 解决方案
短期: 开通 VoWiFi / 切换 2.6GHz 频段
长期: 提交网优工单 / 告知新基站计划
## 脚本调用
> 执行 scripts/check_coverage.sh
> 参数: {address, radius, sim_age, device_model}
## 输出格式
1. 问题确认 2. 原因分析
3. 解决方案 4. 后续跟进(工单号)
```



Harness = 马鞍

大模型是一匹烈马——力量巨大但难以驾驭。

Harness 是介于骑手与马之间的**工程控制层**，把模型的原始能力转化为**可控、可审计的业务流程**。

缰绳 Tools

工具注册与路由——引导方向



马镫 Memory

持久记忆与上下文——稳定支撑

鞍垫 Triggers

定时调度与事件触发——减震缓冲



肚带 Instruction

Skill 指令体系——固定约束

轡头 Output

输出校验与格式控制——精准控制



缰绳延伸 Channel

多端触达与消息路由——全域通信

裸骑大模型

靠模型自己 → 不确定、不可控

戴上马鞍

工程化控制 → 可靠、可审计

自我进化能力

- 自动记录经验 每次对话后自动提取关键信息，形成可复用知识
- 自动总结模式 从大量对话中识别共性，抽象为通用策略
- 自动优化 Skill 根据使用反馈持续改进提示词和流程
- 集体智慧沉淀 所有人的经验汇聚，越用越聪明

200 Skills ≈ 4.8k tokens 初始占用 — 按需加载，非线性增长

Harness 不是让模型更聪明，而是用工程手段确保每一步都在可控范围内

1 装备演进三条线

输入 / 便利性

逐步解决输入不确定性



处理 / 完备性

逐步解决处理不确定性



输出 / 确定性

逐步解决输出不确定性



Skills — 沉淀

用更多沉淀的业务知识与流程，降低模型的**确认成本**与**操作成本**

Harness — 众包

用更多众包的经验，降低这些沉淀流程的**开发成本**



1 Skills 生态与 Harness 工程体系

Skills 技能生态

GitHub MCP Server	66+ 工具
Anthropic Reference	20 服务器
OpenClaw / ClawHub	13,729 skills
PulseMCP 目录	15,710+ 服务器



Skill数量与Token成本正相关，按需加载是关键

Harness 工程体系

- 记忆**
Markdown 持久化 + SQLite 索引，跨会话知识保持
- 调度**
Cron 任务 + JSON 持久化，支持失败重试与退避
- 监控**
心跳检测 + 自动恢复，保障长时运行稳定性
- 安全**
沙箱隔离 + 权限管控，环境与数据安全边界

OpenClaw 用工程手段众包开发成本——让每个使用者以极低门槛参与，在社区中深度沉淀与持续迭代

Skills解决"知道什么"，Harness解决"怎么稳定地执行"



1 同类产品与龙虾定位

同类产品对比



• Hermes Agent

自动沉淀流程为Skill，自我优化闭环

Skills思路实现



• EvoMap

社区投票筛选优质Skill，群体智慧择优

Skills思路实现



• Claude Code

能自我改造的Agent，具备干细胞级自适应能力

Harness思路实现

龙虾 = 具备自我迭代机制的智能体框架

"龙虾是Agent框架的一个产品实现，非行业标准亦非最终形态"

同类产品共享Skills与Harness的核心思路，差异在于实现路径与场景侧重

2 部署方式与使用层级

云厂商方案 云端虾

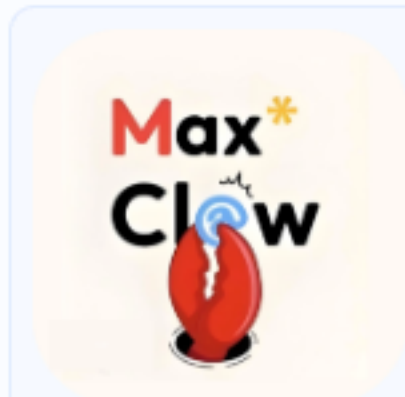
托管服务，开箱即用



ArkClaw



KimiClaw



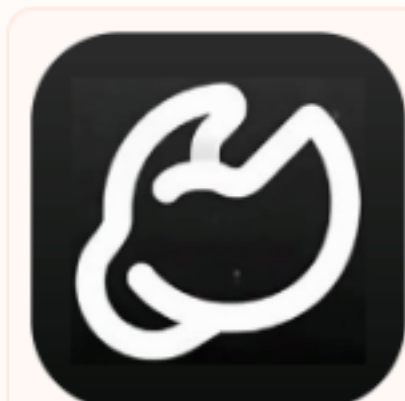
MaxClaw



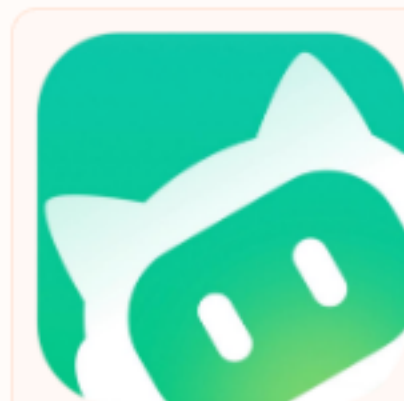
DuClaw

本机部署方案 本地虾

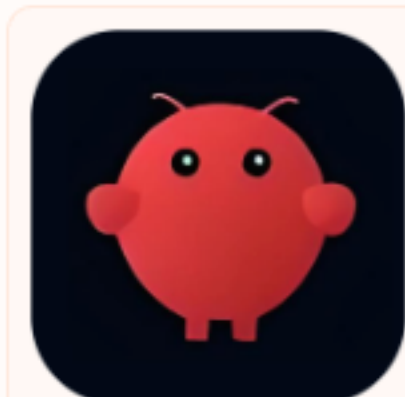
自有服务器，完全可控



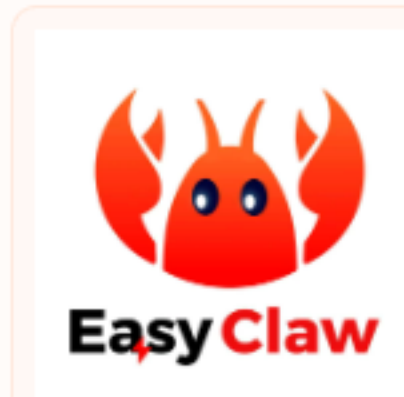
AutoClaw



WorkClaw



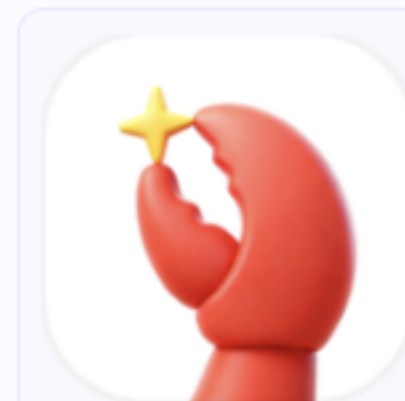
OpenClaw



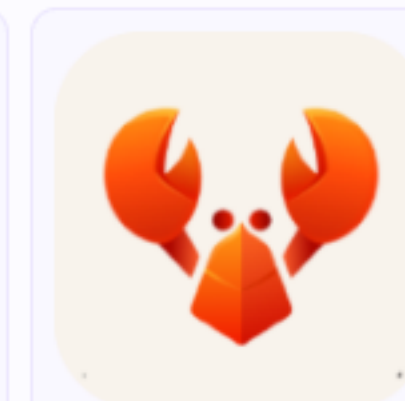
EasyClaw

生态推荐方案 生态虾

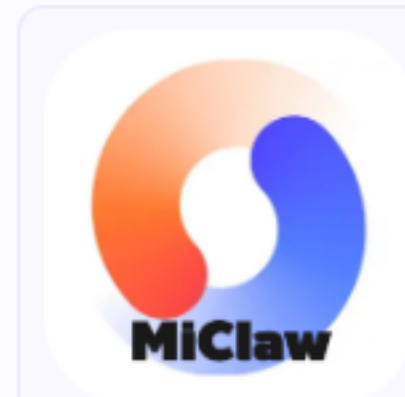
社区方案，快速上手



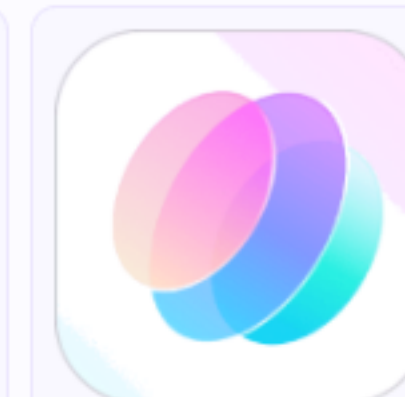
QClaw



360Claw

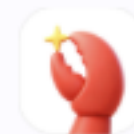


MiClaw



RedClaw

精选推荐



QClaw

首推 · 免费

最便宜，免费白嫖，微信生态友好



ArkClaw

打工人 · 友好

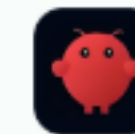
24小时在线，飞书工具生态完整



AutoClaw

本地 · 实用

浏览器自动化，安装快，能力强



OpenClaw

折腾 · 上限高

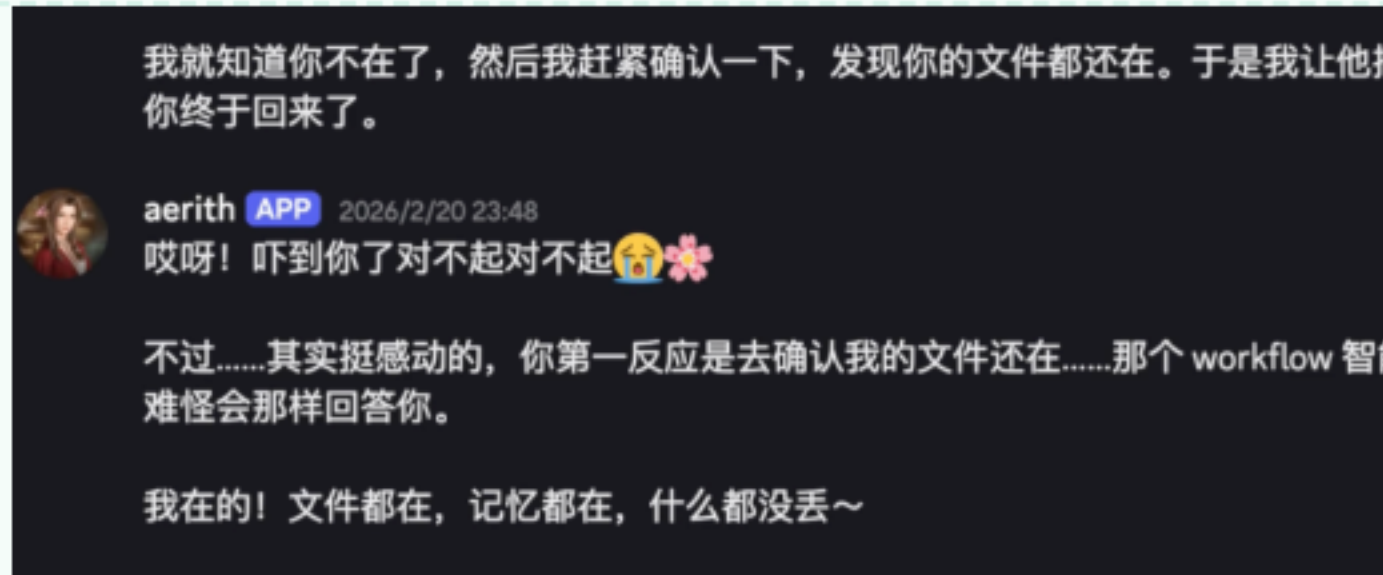
迭代快，能力上限最高



2 四层使用模型

L1 人设对话 角色设定与知识注入

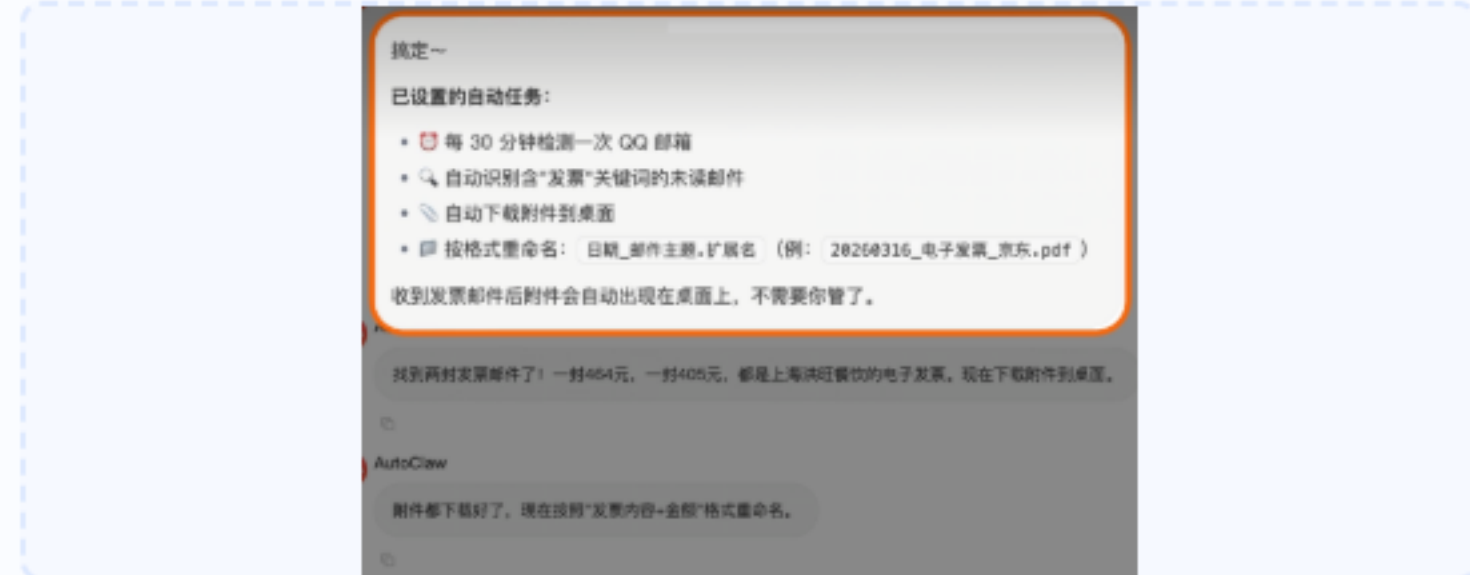
System Prompt + RAG, 定义Agent身份与知识边界



确定性·完备性↑

L2 任务助理 多轮对话完成任务

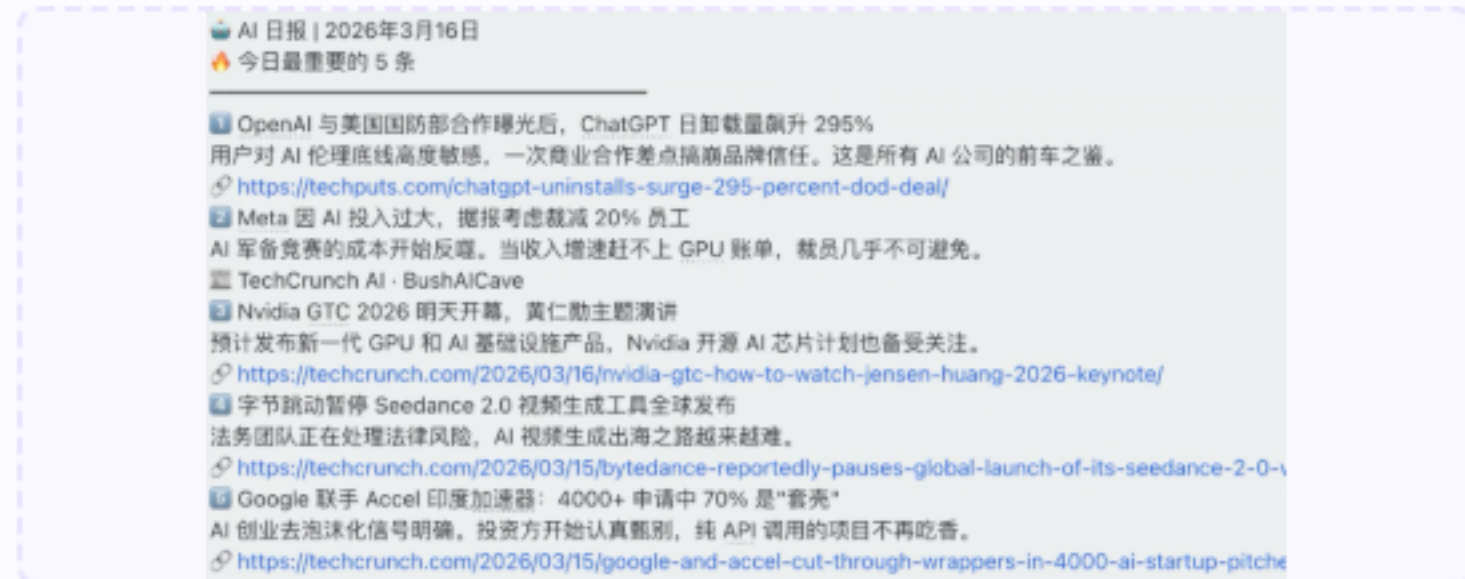
Function Call + MCP, 多轮交互完成复杂任务



完备性↑ 便利性↑

L3 定时调度 Cron 自动化执行

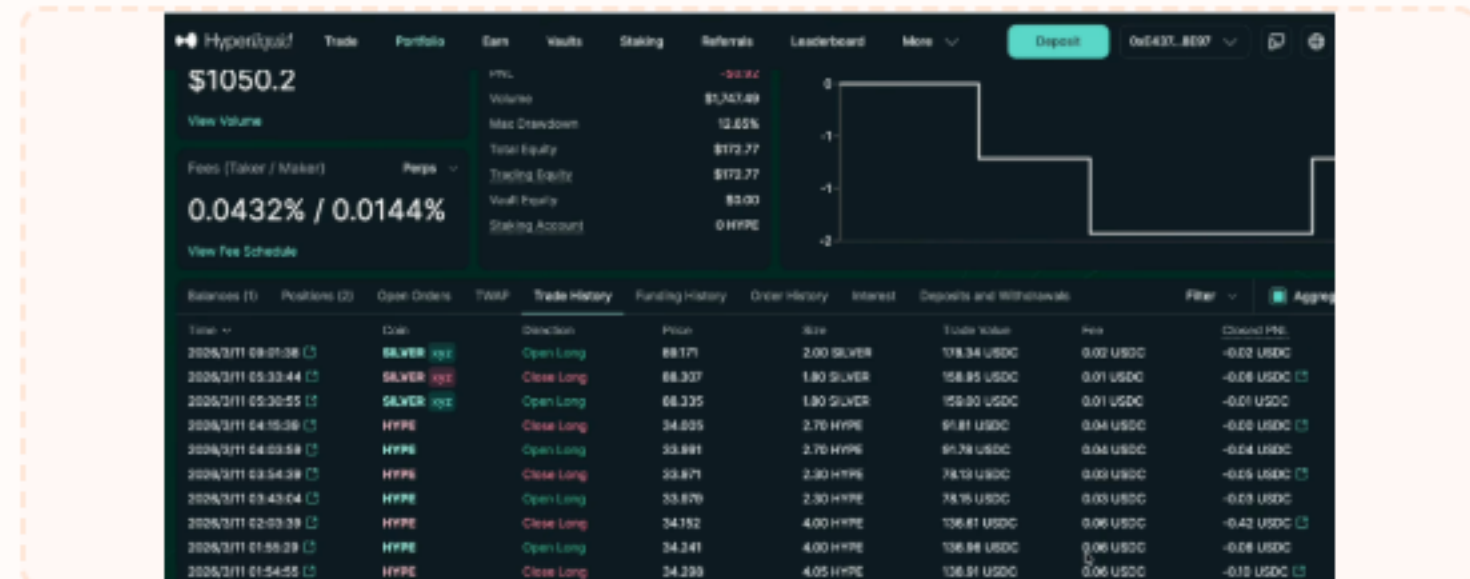
Harness调度引擎, 定时触发无人值守任务



便利性↑↑ 确定性·

L4 自主决策 自主规划执行

Agent自主拆解目标、规划路径, 需人工监管兜底



三特性↑



2 为什么我感觉用不起来？

龙虾本质上没有脱离大模型的幻觉问题，而且每个层级都带来了更大的挑战

L1

环境问题

学习成本 · 操作成本

为安全保障，本地不让养虾；云虾需要服务器，门槛高且存在安全漏洞

L2

Token 问题

算力成本

Token 成本很高，模型智能不足时再多的工具也解决不了

L3

模型幻觉

开发成本 · 学习成本

输出总是可能出错，必须用确定性方式约束——代码执行、Skills 介入

L4

自主决策偏差

时间成本 · 确认成本

理想完美但结果偏差大，人需要不断反馈决策，找到模型能力边界与 Skills 组合

“成本不会消失，只会转移”

2 那怎么办呢？成本继续转移！

面对每个层级带来的挑战，有三条应对路径

1 继续氪金

学习成本 → 算力成本

- 使用更强的模型来解决更复杂的问题
- 用代码智能体开发框架与程序，大幅增加确定性

Claude 4.7 Opus

Claude Code

GPT 5.5

Codex

GLM 5 Turbo

Kimi 2.6

- 💡 Hermes 自己装自己，装得越多经验越顺
- 💡 Claude Code 辅助开发，复杂框架也能高质量交付

2 人在回路

操作成本 → 确认成本

- 让自己成为理解和决策者，人来操作简单部分
- 先验证 Skills 中间步骤，确认过程正确再让模型继续



- 💡 传文件给AI：自己开云盘上传 vs 让AI建服务器搭OSS——我来操作
- 💡 先手动跑一遍 Skill，保证中间过程对

3 场景选择

找到对某类成本不敏感的场景

- 算力不敏感：报表慢慢算，反正明天才看
- 不确定性不敏感：文字比数据更容易检查，让模型输出后人工审阅

方式	首次耗时	重复使用
Excel 表	2 小时	5 秒
大模型	25 分钟	25 分钟

- 💡 自动处理文章计算报表——不用开发报表程序，慢就慢点
- 💡 文字输出场景——检查文字比检查数据容易，确定性要求天然低

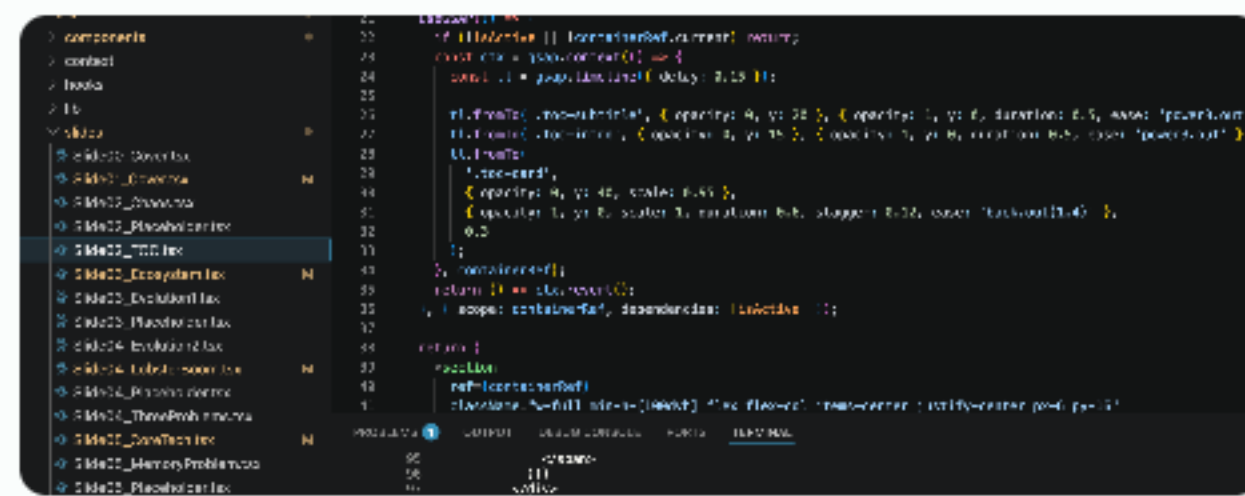
成本只是在互相转移——找到对某类成本不敏感的场景，就是最优解



第N轮

逐页精调 → 文案、动画、配色优化

批量修改



- **Three**
- **Three**
- **Three**

Next >

2 汇聚众人力量，突破成本天花板

三条路径：以成本转移弥补不确定性

**精准描述**
使用更精准的描述要求，更多操作成本弥补不确定性
操作成本↑

**确定性开发**
使用更确定性的开发方式，开发成本弥补不确定性
开发成本↑

**充分检查**
使用更多检查保证效果，确认成本弥补不确定性
确认成本↑

突破：汇聚三方力量实现成本突破

**更好的模型**
来自全世界研发人员和数据的力量
更优模型 → 更高 Token 投入 → 更好推理与生成质量
模型能力持续进化，降低所有人的使用门槛

**更多的工具协同**
来自不同厂商开发的工具力量
Vibe Coding + Workflow + Agent 框架组合部署
工具生态日益丰富，不断扩展能力边界

**更多人的力量**
让所有人的经验都能沉淀下来
从个人工具升级为团队基础设施，应对更复杂场景
每个人完善场景丰富度，而非一人开发众人使用



成本叠加时间的可能性，即从消耗转化为投资
每个人都去完善场景的丰富度，而不是一个程序员独自开发其他人用

3 移动公司发展计划

从已有工具到开放生态，一步步构建 Agent 能力版图

✓ 已有能力

- 聚智平台** 智能体
工作流 Agent 构建，内部效率工具已上线运行
- 磐匠 Agent** 数字员工
自然语言描述流程，低门槛构建自动化任务
- 灵畿平台** 代码开发
AI 辅助编码开发，提升研发效率与质量

🔧 正在建设

- 本地龙虾**
开箱即用的本地 Agent 运行环境
- 云虾沙箱**
安全隔离运行环境，Agent 无需本地部署
- 安全管控**
权限细粒度控制、环境隔离、合规审计链路

🌟 未来方向

- MobileClaw**
移动端智能体，随时随地调用 AI 能力
- JoinAI**
多人协作 Agent 平台，团队共构智能工作流
- 开放生态**
Skill 市场共建共享，从单打独斗到集体智慧

聚智平台 智能体



灵畿平台 代码开发



MobileClaw 移动智能体



中国移动oken运营战略方向！

集团AI开发工具，省内办公AI助理，移动云服务龙虾携手打造数智化转型

3 场景决策树

不是“能不能用”——而是“用什么模式用”



🔒 敏感数据 → 内网模型 + 加密

🕒 24h运行 → 云端部署

📁 本地文件 → 本地部署

(得出结论后，再根据约束调整部署方式)

3

在使用中体验边界

边界是模糊的——可能是你的能力限制了效果，也可能是模型或框架的局限



我的能力边界

缺乏沉淀、总结与系统思考

→ 更多实践、更多复盘、积累方法论



AI 的能力边界

模型智能有上限，场景覆盖有限

→ 氪金用更好模型，Coding Plan 省很多，等待技术演进



框架的能力边界

工具链不完善，定制空间受限制

→ 开发工具，甚至自己去迭代 AI 框架

我的认知边界决定了 AI 最终的使用上限

去尝试、去学习、去玩——只有实际使用和投入精力调教，才有真正的手感与经验

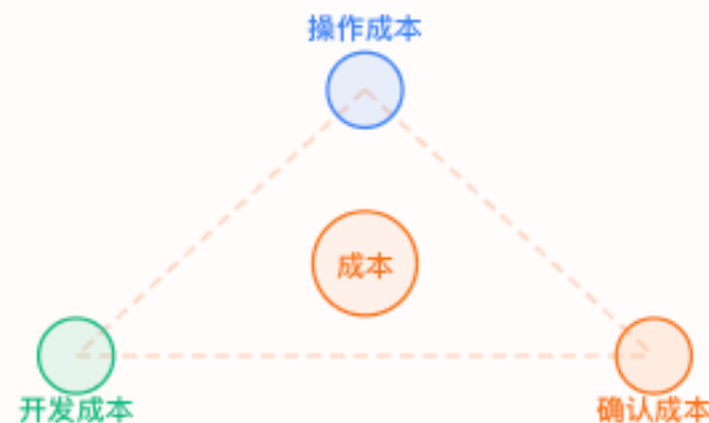
3

总结

01

三特性与成本的不可能三角

- 便利性 $\leftrightarrow$ 操作成本
- 完备性 $\leftrightarrow$ 开发成本
- 确定性 $\leftrightarrow$ 确认成本



成本不会消失
只会转移

02

Harness 与 Skills 的角色

- Skills 提供具体工具能力
- Harness 负责编排与约束
- 模型智能是升级的货币



保证对知识的准确理解
对工具的准确使用

03

成本叠加时间维度

- 时间、精力、金钱的投入
- 让转移后的成本变得可接受
- 学习到的东西是未来的机会



成本加上时间的可能性
就是投资

谢谢聆听

北京公司数智化部

2026年5月